

# CHAPTER – 4

## ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

### Introduction

The **Myntra Fashion E-Commerce Management System** database is designed to manage customers, fashion products, brands, categories, shopping carts, orders, payments, wishlists, reviews, and shipment details efficiently. Functional dependencies define how attributes are related within each table, ensuring that every piece of information is stored in the correct place. Redundancy analysis helps reduce unnecessary duplication of data, while anomaly analysis identifies potential insert, update, and delete issues that may affect data consistency. A well-structured and normalized database improves accuracy, maintains data integrity, minimizes storage redundancy, and supports efficient management of the online fashion shopping platform.

## 1. CUSTOMER TABLE

### Functional Dependency

**Customer\_ID → First\_Name, Last\_Name, Email, Phone\_Number, Password, Gender, Address**

#### Explanation:

The **Customer\_ID** uniquely identifies each customer in the system. All other customer details such as first name, last name, email, phone number, password, gender, and address are functionally dependent on the Customer\_ID. This ensures that every customer record is unique and can be accessed without ambiguity.

## Redundancy Analysis

Customer information is stored only once in the **Customer** table. Other tables, such as **Cart**, **Order**, **Wishlist**, and **Review**, store only the **Customer\_ID** as a foreign key instead of repeating customer details. This minimizes duplicate data, reduces storage requirements, and maintains consistency throughout the database.

## Data Anomalies

- **Insert Anomaly**

A new customer can register and have their details stored in the Customer table even if they have not placed an order or added any product to their cart or wishlist.

- **Update Anomaly**

If a customer changes their phone number, email address, or delivery address, the information needs to be updated only once in the Customer table. All related tables automatically refer to the updated information through the Customer\_ID.

- **Delete Anomaly**

Deleting a customer's order, wishlist item, or review does not remove the customer's personal information from the Customer table. The customer's account remains available for future purchases.

## 2. BRAND TABLE

### Functional Dependency

**Brand\_ID** → **Brand\_Name**, **Country**, **Description**

#### Explanation

The **Brand\_ID** uniquely identifies each brand available on the Myntra platform. All other brand details, including the brand name, country of origin, and description, are

functionally dependent on the **Brand\_ID**. This ensures that each brand has a unique identity and prevents confusion between different brands.

## Redundancy Analysis

Brand information is stored only once in the **Brand** table. The **Product** table stores only the **Brand\_ID** as a foreign key instead of repeating the brand name, country, and description for every product. This reduces data duplication, saves storage space, and ensures that all products belonging to the same brand refer to a single, consistent brand record.

## Data Anomalies

- **Insert Anomaly**

A new brand can be added to the Brand table even if no products are currently available under that brand.

- **Update Anomaly**

If a brand changes its name, description, or country information, the update is made only once in the Brand table. All products associated with that brand automatically reflect the updated information through the **Brand\_ID**.

- **Delete Anomaly**

Deleting a product from the Product table does not remove the corresponding brand information. The brand record remains in the database and can be used for future products.

## 3. CATEGORY TABLE

### Functional Dependency

**Category\_ID** → **Category\_Name**, **Description**

#### Explanation

The **Category\_ID** uniquely identifies each product category in the Myntra Fashion E-Commerce Management System. Every category has a unique name and description, which are functionally dependent on the **Category\_ID**. This ensures that each category is stored only once and can be referenced by multiple products without repeating category information.

### Redundancy Analysis

Category information is maintained only in the **Category** table. The **Product** table stores only the **Category\_ID** as a foreign key instead of repeating the category name and description for every product. This minimizes data redundancy, improves storage efficiency, and ensures consistency across the database.

### Data Anomalies

- **Insert Anomaly**

A new category, such as **Sportswear**, **Ethnic Wear**, or **Footwear**, can be added to the Category table even if no products currently belong to that category.

- **Update Anomaly**

If the name or description of a category needs to be modified, the change is made only once in the Category table. All products associated with that category automatically reflect the updated information through the **Category\_ID**.

- **Delete Anomaly**

Deleting a product from the Product table does not remove its corresponding category. The category remains available in the database for future products.

## 4. PRODUCT TABLE

### Functional Dependency

**Product\_ID** → **Product\_Name**, **Brand\_ID**, **Category\_ID**, **Size**, **Color**, **Price**, **Stock**, **Description**

#### Explanation

The **Product\_ID** uniquely identifies each product available in the Myntra Fashion E-Commerce Management System. All product details, such as the product name, brand, category, size, color, price, stock quantity, and description, are functionally dependent on the **Product\_ID**. This ensures that every product has a unique identity and prevents duplicate product records in the database.

### Redundancy Analysis

Product information is stored only once in the **Product** table. Instead of storing complete product details in tables such as **Order\_Item**, **Wishlist**, and **Review**, only the **Product\_ID** is referenced as a foreign key. Similarly, the **Brand\_ID** and **Category\_ID** are used to link the product with its corresponding brand and category. This design reduces data duplication, saves storage space, and maintains consistency throughout the database.

## Data Anomalies

- **Insert Anomaly**

A new product can be added to the Product table before it is purchased, reviewed, or added to a customer's wishlist. As long as the associated Brand\_ID and Category\_ID exist, the product record can be inserted successfully.

- **Update Anomaly**

If the price, stock quantity, description, or color of a product changes, the update is made only once in the Product table. All related tables automatically refer to the updated information through the Product\_ID, ensuring consistency across the system.

- **Delete Anomaly**

Deleting an order, wishlist entry, or customer review does not remove the product information from the Product table. The product remains available in the database and can continue to be displayed for future customers unless it is intentionally removed by the administrator.

## 5. CART TABLE

### Functional Dependency

**Cart\_ID → Customer\_ID, Total\_Items, Grand\_Total**

#### Explanation

The **Cart\_ID** uniquely identifies each shopping cart in the Myntra Fashion E-Commerce Management System. Each cart belongs to a specific customer through the **Customer\_ID**. The total number of products in the cart (**Total\_Items**) and the overall cart value (**Grand\_Total**) are associated with the Cart\_ID. This ensures that every shopping cart has its own unique record and can be managed efficiently.

## Redundancy Analysis

The **Cart** table stores only cart-related information. Customer details are not repeated because they are linked through the **Customer\_ID**. The **Grand\_Total** is calculated based on the products added to the cart, preventing unnecessary duplication of product information. This design improves storage efficiency and keeps the database consistent.

## Data Anomalies

- **Insert Anomaly**

A shopping cart can be created for a registered customer even if no products have been added yet. Initially, the **Total\_Items** and **Grand\_Total** may be zero.

- **Update Anomaly**

Whenever a customer adds or removes a product from the cart, only the **Total\_Items** and **Grand\_Total** values need to be updated in the Cart table. No changes are required in the Customer or Product tables.

- **Delete Anomaly**

Deleting all products from a shopping cart does not remove the customer's account or product records from the database. The cart remains available for future shopping activities.

## 6. ORDER TABLE

### Functional Dependency

**Order\_ID** → **Customer\_ID**, **Order\_Date**, **Order\_Status**, **Total\_Amount**

## Explanation

The **Order\_ID** uniquely identifies each order placed in the Myntra Fashion E-Commerce Management System. Every order is associated with a specific customer through the **Customer\_ID**. The order date, current order status, and total amount are all functionally dependent on the **Order\_ID**. This ensures that each order has a unique record and all order-related information is maintained accurately.

## Redundancy Analysis

The **Order** table stores only order-related information. Customer details are not duplicated because they are referenced using the **Customer\_ID**. Product information is stored separately in the **Order\_Item** table, while payment and shipment details are maintained in their respective tables. This normalization reduces data redundancy, improves storage efficiency, and ensures data consistency throughout the database.

## Data Anomalies

### • Insert Anomaly

A new order can be created for an existing customer using the **Customer\_ID**. The order record can be inserted before payment or shipment details are added.

### • Update Anomaly

If the order status changes (for example, from **Processing** to **Shipped** or **Delivered**), the update is made only once in the Order table. Similarly, any correction to the total amount is updated in a single record without affecting other tables.

### • Delete Anomaly

Deleting an order removes only the order record. Customer information, product details, payment records, and shipment details stored in their respective tables remain unaffected unless they are explicitly deleted according to database rules.

## 7. ORDER\_ITEM TABLE

### Functional Dependency

**OrderItem\_ID** → **Order\_ID**, **Product\_ID**, **Quantity**, **Unit\_Price**

#### Explanation

The **OrderItem\_ID** uniquely identifies each item included in an order. Every record in the **Order\_Item** table is associated with a specific order through the **Order\_ID** and a particular product through the **Product\_ID**. The quantity of the product ordered and its unit price are functionally dependent on the **OrderItem\_ID**. This allows multiple products to be included in a single order while maintaining accurate details for each product.

### Redundancy Analysis

The **Order\_Item** table acts as a bridge between the **Order** and **Product** tables. Instead of storing complete product information within the Order table, only the **Product\_ID** is stored along with the quantity and unit price. This prevents duplication of product details and ensures that each ordered product is recorded efficiently. The table also supports multiple products in a single order without repeating order information.

### Data Anomalies

- **Insert Anomaly**

A new order item can be added only after a valid order has been created and an existing product has been selected. The record requires both **Order\_ID** and **Product\_ID** to maintain referential integrity.

- **Update Anomaly**

If the quantity of a product in an order changes before the order is confirmed, only the corresponding record in the **Order\_Item** table needs to be updated. The product details and order information remain unchanged.

- **Delete Anomaly**

Deleting an item from the **Order\_Item** table removes only that particular product from the order. It does not delete the product from the **Product** table or remove the complete order from the **Order** table.

## 8. PAYMENT TABLE

### Functional Dependency

**Payment\_ID** → **Order\_ID**, **Payment\_Method**, **Payment\_Status**, **Payment\_Date**, **Amount**

#### Explanation

The **Payment\_ID** uniquely identifies each payment transaction in the Myntra Fashion E-Commerce Management System. Every payment is associated with a specific order through the **Order\_ID**. The payment method, payment status, payment date, and payment amount are all functionally dependent on the **Payment\_ID**. This ensures that each payment record is unique and accurately linked to its corresponding order.

### Redundancy Analysis

The **Payment** table stores all payment-related information separately from the **Order** table. Instead of repeating payment details in every order record, only the **Order\_ID** is used to establish the relationship. This reduces data duplication, simplifies payment management, and ensures that payment information is maintained consistently across the database.

## Data Anomalies

- **Insert Anomaly**

A payment record can be created only after a valid order has been placed. The payment details, such as payment method and amount, are then recorded using the corresponding **Order\_ID**.

- **Update Anomaly**

If the payment status changes (for example, from **Pending** to **Successful** or **Failed**), the update is made only once in the **Payment** table. This ensures that payment information remains accurate without affecting order details.

- **Delete Anomaly**

Deleting a payment record does not remove the corresponding order from the **Order** table. The order details remain available, allowing administrators to maintain order history and manage payment issues separately.

## 9. WISHLIST TABLE

### Functional Dependency

**Wishlist\_ID** → **Customer\_ID**, **Product\_ID**, **Date\_Added**

#### Explanation

The **Wishlist\_ID** uniquely identifies each wishlist entry in the Myntra Fashion E-Commerce Management System. Every wishlist record is associated with a specific customer through the **Customer\_ID** and a specific product through the **Product\_ID**. The date on which the product was added to the wishlist is also functionally dependent on the **Wishlist\_ID**. This allows customers to save products for future purchase without affecting their shopping cart or order history.

## Redundancy Analysis

The **Wishlist** table stores only wishlist-related information. Customer details are maintained in the **Customer** table, while product information is stored in the **Product** table. The Wishlist table contains only the **Customer\_ID**, **Product\_ID**, and **Date\_Added**, avoiding duplication of customer and product details. This design improves storage efficiency and maintains consistency across the database.

## Data Anomalies

- **Insert Anomaly**

A customer can add a product to the wishlist only if both the customer and the product already exist in the database. A new wishlist entry is created using the corresponding **Customer\_ID** and **Product\_ID**.

- **Update Anomaly**

If the date associated with a wishlist entry needs to be corrected, only the corresponding record in the **Wishlist** table is updated. Customer and product information remain unchanged.

- **Delete Anomaly**

Deleting a wishlist entry removes only that product from the customer's wishlist. It does not delete the customer account or remove the product from the **Product** table.

## 10. REVIEW TABLE

### Functional Dependency

**Review\_ID** → **Customer\_ID**, **Product\_ID**, **Rating**, **Review\_Text**, **Review\_Date**

## Explanation

The **Review\_ID** uniquely identifies each customer review in the Myntra Fashion E-Commerce Management System. Every review is associated with a specific customer through the **Customer\_ID** and a specific product through the **Product\_ID**. The rating, review text, and review date are all functionally dependent on the **Review\_ID**. This ensures that each review is uniquely stored and can be easily managed.

## Redundancy Analysis

The **Review** table stores only review-related information. Customer details are maintained in the **Customer** table, while product details are stored in the **Product** table. Instead of repeating customer names or product information, the Review table stores only the **Customer\_ID** and **Product\_ID**. This reduces data redundancy, improves storage efficiency, and maintains consistency throughout the database.

## Data Anomalies

### • Insert Anomaly

A customer can submit a review only if both the customer and the product already exist in the database. A new review is created using the corresponding **Customer\_ID** and **Product\_ID**.

### • Update Anomaly

If a customer wants to edit their rating or update the review text, only the corresponding record in the **Review** table needs to be modified. Customer and product details remain unchanged.

### • Delete Anomaly

Deleting a review removes only the review record from the **Review** table. It does not delete the customer account or remove the product from the **Product** table.

# 11. SHIPMENT TABLE

## Functional Dependency

**Shipment\_ID** → **Order\_ID**, **Delivery\_Address**, **Shipment\_Date**, **Delivery\_Date**, **Shipment\_Status**

### Explanation

The **Shipment\_ID** uniquely identifies each shipment in the Myntra Fashion E-Commerce Management System. Every shipment is linked to a specific order through the **Order\_ID**. The delivery address, shipment date, delivery date, and shipment status are all functionally dependent on the **Shipment\_ID**. This ensures that shipment details are accurately maintained and can be tracked independently for each order.

## Redundancy Analysis

The **Shipment** table stores only shipment-related information. Instead of repeating delivery details in the **Order** table, the shipment record is linked using the **Order\_ID**. This reduces data redundancy, simplifies shipment tracking, and ensures that delivery information is stored consistently without unnecessary duplication.

## Data Anomalies

### • Insert Anomaly

A shipment record can be created only after a valid order has been confirmed. The shipment details are then recorded using the corresponding **Order\_ID**.

### • Update Anomaly

If the shipment status changes (for example, from **Packed** to **Shipped**, **Out for Delivery**, or **Delivered**), the update is made only once in the **Shipment** table. This keeps delivery information accurate without affecting order or customer records.

- **Delete Anomaly**

Deleting a shipment record removes only the shipment details. The corresponding order, customer information, and payment records remain available in their respective tables unless they are explicitly deleted.

## **CONCLUSION**

The **Myntra Fashion E-Commerce Management System** is designed using proper functional dependencies and normalized tables to ensure efficient data management. The database structure reduces data redundancy, maintains data consistency, and prevents insert, update, and delete anomalies. Overall, the system provides an organized, reliable, and efficient solution for managing customers, products, orders, payments, and shipments in an online fashion e-commerce platform.